PRIVSIGHT · TRACERA

Full Session Handoff & Technical Architecture Dossier

Trace. Analyze. Understand. — AI-Powered Relationship Intelligence Assistant

Generated: 2026-10-31 00:56:45	**Target Branch:** main	**Latest Commit:** 107e7bb
Repository: aiori-hackathon/A3-PS019-TC303	**Dev Server:** http://127.0.0.1:8000	**Status:** Clean & Verified (11/11 tests pass)

---

1. Executive Summary & Purpose

This dossier contains the complete context, architectural blueprints, bug resolution history, and conversation logs of the **TRACERA AI Assistant** within the **PrivSight** privacy governance platform. A new chat instance can ingest this document to immediately resume development without losing architectural context or repeating previous diagnostic cycles.

Core Product Identity: TRACERA (*Trace. Analyze. Understand.*) is PrivSight's native AI relationship intelligence assistant. It provides zero-fabrication relational reasoning across scanned CSVs, SQLite databases, and multi-format files, distinguishing confirmed foreign/primary keys from confidence-scored inferences with strict PII masking.

2. Chronology of Completed Tasks

- **Milestone 1: Terminology Normalization:** Updated legacy labels to exact standardized enterprise terms: 'Data Fields Checked' (from Fields Scanned), 'Personal Data Found' (from PII Candidates), 'Metadata Completed' (from Complete Metadata), and 'Missing Proof' (from Evidence Gaps) without altering business logic.
- **Milestone 2: Relationship Explorer Engine:** Built end-to-end backend and frontend relationship engine in backend/relationships/: relationship graph models, schema constraint discovery, cross-dataset intersection heuristics, Cytoscape.js interactive topology graph, and CSV/SQLite multi-source scanner.
- **Milestone 3: Dedicated TRACERA Page Integration:** Integrated TRACERA as a top-level dedicated platform page at /tracera inside PrivSight, featuring persistent threads, sidebar history, dataset filter pill trays, format preference selectors (Text, Graph, Table, Timeline, Flowchart), and disclaimer banners.
- **Milestone 4: Chat Message Alignment & Deduplication Fix (Highest Priority):** Completely re-engineered message lifecycle: resolved duplicate message rendering via client-side Set deduplication, added request mutex locks, fixed context inheritance bugs where queries inherited previous entities, applied strict ChatGPT-style right-alignment for user bubbles and left-alignment for TRACERA AI cards, and implemented full Markdown rendering for tables and code blocks.

3. Root Causes & Technical Solutions Implemented

Issue Category	Root Cause Diagnosed	Architectural Solution Applied
Message Alignment	User and assistant messages shared the same row wrapper with no strict flexbox justification.	Created distinct CSS classes: .tracera-message-row.user (flex-end, 75% max width) and .tracera-message-row.assistant (flex-start, full width card with avatar and header).
Conversation ID Disconnect	Backend created new conversations on first prompt, but RelationshipReport did not serialize conversation_id back to frontend.	Added conversation_id and message_id fields to RelationshipReport in models.py and routes.py. Client locks onto conversation ID immediately.
Duplicate Rendering	Missing idempotency check during DOM rendering and missing mutex lock on prompt submissions.	Implemented renderedMessageIds = new Set() in ChatMessageList and an isSubmitting guard in MessageComposer.
Repeated Answers	In ai_assistant.py line 177, non-ID queries fell back to previous history entity even without pronouns.	Constrained context inheritance strictly to pronoun-based queries (has_pronoun and history).
Welcome Screen Flaws	Welcome hero was rendered inside messages container and reappeared erratically.	Encapsulated WelcomeScreen component: displays ONLY when message count is 0; destroyed on first send; restored on '+ New Chat'.
Markdown Formatting	Markdown rendered as raw unformatted text without table, list, or code support.	Engineered Markdown parser supporting tables (.tracera-md-table), code blocks (pre code), blockquotes, headings h1-h4, and lists.

4. Key Files & Architecture Map

File Path	Component / Purpose
`frontend/tracera.js`	Client-side modular controller: WelcomeScreen, UserMessage, AssistantMessage, ChatMessageList, MessageComposer, Markdown engine, Cytoscape graph renderer.
`frontend/tracera.html`	Dedicated HTML page layout for TRACERA assistant, sidebar history, dataset filter tray, messages container.
`frontend/style.css`	PrivSight design system tokens, flexbox alignment for user/assistant messages, markdown typography, table and code styles.
`backend/relationships/ai_assistant.py`	TRACERA AI reasoning engine: query intent detection, zero-fabrication factual retrieval, multi-turn context resolution.
`backend/relationships/models.py`	Pydantic models: RelationshipReport, ChatMessage, ConversationThread, RelationshipGraph.
`backend/relationships/routes.py`	FastAPI endpoints: /api/relationships/chat, /conversations, /sources, /graph.
`tests/test_relationships.py`	11 automated backend integration tests verifying auth, lifecycle, context, graph, table, timeline, and titling.

5. Quality Assurance & Test Verification Log

All 10 required prompt test scenarios were executed and passed:

Test Case	Verification Method	Status
TEST 1: User message right alignment	Single message prompt sent; verified justify-content: flex-end on .tracera-message-row.user.	**PASSED**
TEST 2: AI response left alignment	AI response rendered via .tracera-message-row.assistant with avatar and branding.	**PASSED**
TEST 3: Alternating multi-turn alignment	Consecutive messages verified: User (R) -> AI (L) -> User (R) -> AI (L).	**PASSED**
TEST 4: Welcome screen lifecycle	Welcome screen disappears on prompt submission; reappears on '+ New Chat'.	**PASSED**
TEST 5: Page refresh deduplication	Refreshed active conversation; verified renderedMessageIds prevented duplicate rendering.	**PASSED**
TEST 6: Thread history loading	Selected older thread; verified chronological message restoration.	**PASSED**
TEST 7: Markdown formatting	Verified headings, code snippets, blockquotes, and tables render with clear vertical spacing.	**PASSED**
TEST 8: New chat isolation	Clicked '+ New Chat'; created isolated thread without old conversation leakage.	**PASSED**
TEST 9: Console & Network sanity	Inspected network calls; 0 duplicate POSTs or render loops.	**PASSED**

TEST 10: Fixed bottom composer	Composer auto-expands from 110px to 250px; Enter key sends; Shift+Enter creates newline.	**PASSED**

6. Chronological Chat Transcript & Handoff Prompts

Below is the record of user prompts and agent actions from this session:

User Prompt (Initial)
analyse the project, run the project and give me the link. Make changes in name of functionalities: Fields Scanned -> Data Fields Checked, PII Candidates -> Personal Data Found, Complete Metadata -> Metadata Completed, Evidence Gaps -> Missing Proof. Push to repo.

Agent Action
Renamed UI labels in frontend/index.html, frontend/app.js, frontend/inventory.html, docs/demo_checklist.md, and verified backend functionality remained intact. Committed and pushed to GitHub.

User Prompt (Relationship Explorer)
Add AI Relationship Explorer into PrivSight: real uploaded dataset analysis, SQLite foreign keys, CSV intersection, interactive graph canvas.

Agent Action
Implemented backend/relationships/ with Graph Engine, Models, Routes, and Frontend visual Cytoscape graph canvas with 11 unit tests passing.

User Prompt (TRACERA Integration)
Integrate TRACERA AI assistant (Trace. Analyze. Understand.) as a dedicated page inside PrivSight. Support persistent threads, format switcher, dataset filtering.

Agent Action
Created frontend/tracera.html, backend/relationships/ai_assistant.py, and automated thread management with zero-fabrication evidence provenance.

User Prompt (Fix Chat Alignment & Duplication)
Fix chat message alignment (user on right, TRACERA AI on left), repeated/duplicated content, welcome screen lifecycle, markdown rendering, modular component architecture.

Agent Action
Completed comprehensive fix across backend models, routes, ai_assistant.py, frontend style.css, tracera.html, and tracera.js. All 10 tests verified in browser. Pushed commit 107e7bb to main.

7. Operational Runbook for the Next Chat Agent

To immediately spin up or verify the environment in a new session:

- **Run Server:** .venv\Scripts\python.exe -m uvicorn backend.api.main:app --host 127.0.0.1 --port 8000
- **TRACERA URL:** http://127.0.0.1:8000/tracera
- **Workspace URL:** http://127.0.0.1:8000/
- **Auth Credentials:** admin / password123 or analyst / password123
- **Run Test Suite:** .venv\Scripts\pytest tests/test_relationships.py -v (11 passing tests)
- **Git Remote:** https://github.com/aiori-hackathon/A3-PS019-TC303.git on branch main